🎓 MIDTERM PROJECT – ANGULAR APPLICATION

🚀 SkillSwap Platform

📌 1. Project Overview

For your midterm project, you must develop a **fully functional Angular frontend application** that integrates with the official SkillSwap REST API.

SkillSwap is a freelance marketplace platform where:

- 👤 Users register and log in
- 💼 Clients post jobs
- 📩 Freelancers submit proposals
- ✅ Clients accept proposals
- 🔄 Jobs move from "open" → "in_progress" → "completed"
- ⭐ Both participants leave reviews
- 📊 User ratings are recalculated automatically

You are responsible for building the **complete frontend application**.

🌍 2. API Base URL

Production Base URL: https://stingray-app-wxhhn.ondigitalocean.app

All endpoints described below are relative to this base URL.

🔐 3. Authentication

Protected endpoints require JWT authentication.

All authenticated requests must include:

Authorization: Bearer <token>

The token is returned by:

POST /auth/login

⚠️ Common Authentication Errors

401 – Missing or invalid Authorization header
401 – Token has expired
401 – Invalid token

Your Angular application must:

✓ Store the JWT after login
✓ Automatically attach it using an HTTP interceptor
✓ Redirect to login page if a 401 error occurs

📦 4. General Response Format

Successful responses return:

- A JSON object
- A JSON array

Error responses always return:

{ "error": "Error message here" }

Common HTTP status codes:

200 – OK
201 – Created
400 – Bad Request
401 – Unauthorized
403 – Forbidden
404 – Not Found
409 – Conflict

Your application must properly handle and display ALL backend error messages.

🔐 AUTH MODULE

📝 POST /auth/register

Creates a new user account.

Required fields:

- name
- username
- email
- password
- bio
- skills (array)

✅ Success (201)

Returns a message and the created user object.

❌ **Possible Errors**

400 – Missing required fields
400 – Invalid username
409 – Email already in use
409 – Username already in use (returns suggested_username)

Your frontend must properly display the suggested username if returned.

🔑 **POST /auth/login**

Authenticates user credentials.

Required:

- email
- password

✅ **Success (200)**

Returns:
- token
- user object

❌ **Possible Errors**

400 – Email and password are required
401 – Invalid credentials

After login, you must store the token securely.

👤 USERS MODULE

🔐 **GET /users/me (Authentication Required)**

Returns the currently authenticated user profile.

❌ **Possible Errors**

404 – User not found
401 – Invalid or expired token

🌐 **GET /users/<username>**

Returns public profile data including:

- id
- name
- username
- bio
- skills
- rating_avg
- completed_jobs

❌ **Possible Errors**

404 – User not found

💼 JOBS MODULE

🔎 POST /jobs/search

Search jobs using filters.

Optional filters (in BODY):

- category
- status (default is “open”)
- min_budget

❌ **Possible Errors**

400 – min_budget must be numeric

Returns an array of job objects.

➕ POST /jobs (Authentication Required)

Creates a new job.

Required:

- title
- description
- budget
- category

❌ **Possible Errors**

400 – Missing required fields
401 – Unauthorized

📄 GET /jobs/<job_id> (Authentication Required)

Returns detailed job information including owner and freelancer summary.

❌ Possible Errors

404 – Job not found
401 – Unauthorized

✏️ PATCH /jobs/<job_id> (Owner Only)

Can update:

- title
- description
- budget
- category
- status

Valid status values:

- open
- in_progress
- completed

❌ Possible Errors

404 – Job not found
403 – Forbidden
400 – Invalid status
400 – No valid fields to update

📋 GET /jobs/my-postings (Authentication Required)

Returns jobs created by logged-in user.

✅ PATCH /jobs/<job_id>/complete (Authentication Required)

Marks job as completed.

Rules:

- Only owner or assigned freelancer
- Job must be in_progress

❌ Possible Errors

404 – Job not found
403 – Forbidden
400 – Only in-progress jobs can be completed

📩 PROPOSALS MODULE

📤 POST /jobs/<job_id>/proposals (Authentication Required)

Submit a proposal.

Required:

- price
- cover_letter OR message

Rules:

- Cannot submit to your own job
- Only open jobs
- Only one pending proposal per job

❌ Possible Errors

404 – Job not found
403 – You cannot submit a proposal to your own job
400 – Proposals are allowed only for open jobs
409 – You already have a pending proposal for this job
400 – price and cover_letter/message are required

📑 GET /jobs/<job_id>/proposals (Owner Only)

❌ **Possible Errors**

404 – Job not found
403 – Forbidden

🟢 **PATCH /proposals/<proposal_id>/accept (Authentication Required)**

Accepts proposal and automatically:

- Rejects other proposals
- Sets job to in_progress

❌ **Possible Errors**

404 – Proposal not found
404 – Related job not found
403 – Forbidden
403 – You cannot accept your own proposal
400 – Job is no longer open

📂 **GET /proposals/my-bids (Authentication Required)**

Returns proposals submitted by user.

🗑 **DELETE /proposals/<proposal_id> (Authentication Required)**

Withdraw proposal.

Rules:

- Only proposal owner
- Only pending proposals

❌ **Possible Errors**

404 – Proposal not found
403 – Forbidden
400 – Only pending proposals can be deleted

⭐ REVIEWS MODULE

✍️ POST /jobs/<job_id>/reviews (Authentication Required)

Submit a review.

Rules:

- Job must be completed
- Reviewer must be owner or freelancer
- Cannot review yourself
- target_id must be other participant
- rating must be integer between 1 and 5
- Only one review per job

❌ Possible Errors

404 – Job not found
400 – Reviews are allowed only for completed jobs
403 – Forbidden
400 – target_id and rating are required
400 – You cannot review yourself
400 – target_id must be the other participant
400 – rating must be an integer between 1 and 5
409 – You already reviewed this user for this job
404 – Target user not found

📥 GET /reviews/user/<user_id>

❌ Possible Errors

400 – Invalid user id

📊 PLATFORM MODULE

📈 GET /platform/stats

Public endpoint.

Returns:

- total_users
- active_jobs
- total_value_moved

👥 6. Group Work Requirements

This project must be completed in groups of **maximum 3 students**.

Requirements:

✓ All members must actively contribute
✓ The Git repository must show commits from ALL students
✓ Work distribution must be balanced

🎥 7. Submission Requirements

You must submit:

The Git repository link
- No node_modules folder
- Clean and organized project

A recorded video presentation (10–15 minutes recommended) including:

- Application overview
- Demonstration of the complete business flow
- Explanation of architecture
- Demonstration of error handling
- Participation of all group members

The video must clearly show:

Register → Login → Post Job → Submit Proposal → Accept Proposal → Complete Job → Leave Reviews → Rating Update

🔁 8. Mandatory Business Flow

The following flow must fully work:

1. User A registers

2. User A logs in
3. User A posts a job
4. User B registers
5. User B submits a proposal
6. User A accepts proposal
7. Job moves to in_progress
8. One participant completes job
9. Both leave reviews
10. Ratings update correctly

🚫 9. Strict Rules

❌ Do NOT hardcode data
❌ Do NOT simulate API responses
❌ Do NOT fake authentication

✓ All data must come from the API
✓ All documented errors must be handled and displayed

🏆 10. Evaluation Criteria

You will be graded based on:

- Correct API integration
- JWT handling
- Error handling
- Form validation
- Angular architecture
- Clean UI/UX
- Business flow completeness
- Code quality and organization
- Equal group contribution
- Quality of video presentation